

Computer Organization and Design

Instructions: Language of the Computer

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
 - But with many aspects in common
- Early computers had very simple instruction sets
 - Simplified implementation
- Many modern computers also have simple instruction sets

The MIPS Instruction Set

- Used as the example throughout the course
- Stanford MIPS commercialized by MIPS Technologies (www.mips.com)
- Large share of **embedded** core market
 - Applications in smartphones, tablets, consumer electronics, network/storage equipment, cameras, printers, ...

Arithmetic Operations

- Add and subtract, three operands
 - Two sources and one destination

add a, b, c # a gets b + c
- All arithmetic operations have this form
- *Design Principle 1*: Simplicity favours regularity
 - Regularity makes implementation simpler
 - Simplicity enables higher performance at lower cost

Arithmetic Example

- C code:

$f = (g + h) - (i + j);$

- Compiled MIPS code:

```
add t0, g, h    # temp t0 = g + h
add t1, i, j    # temp t1 = i + j
sub f, t0, t1   # f = t0 - t1
```

Register Operands

- Arithmetic instructions use **register operands**
- MIPS has a **32 × 32**-bit register file
 - Use for frequently accessed data
 - Numbered 0 to 31
 - 32-bit data called a “word”
- Assembler names
 - **\$t0, \$t1, ..., \$t9** for temporary values
 - **\$s0, \$s1, ..., \$s7** for saved variables
- ***Design Principle 2***: Smaller is faster
 - Reason for small register file
 - c.f. main memory: millions of locations

Register Operand Example

- C code:

$f = (g + h) - (i + j);$

- $f, \dots, j$ in $\$s0, \dots, \$s4$

- Compiled MIPS code:

`add $t0, $s1, $s2`

`add $t1, $s3, $s4`

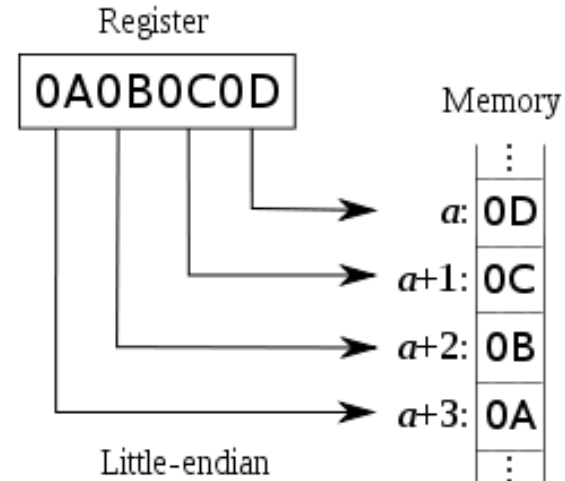
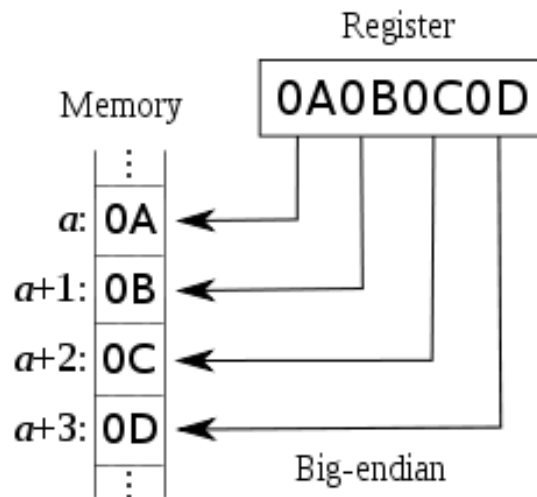
`sub $s0, $t0, $t1`

Memory Operands

- Main memory used for composite data
 - Arrays, structures, dynamic data, ...
 - Only a small amount of data can fit in registers
- To apply arithmetic operations
 - Load values from memory into registers
 - Store result from register to memory
- Memory is **byte** addressed
 - Each address identifies an 8-bit byte
- Words are aligned in memory
 - Address must be a multiple of 4
- MIPS is **Big Endian**
 - Most-significant byte at least address of a word
 - *c.f.* Little Endian: least-significant byte at least address

Big vs. Little Endian

- ❑ **Big Endian:** leftmost byte is word address
IBM 360/370, Motorola 68k, **MIPS**, Sparc, HP PA
- ❑ **Little Endian:** rightmost byte is word address
Intel 80x86, DEC Vax, DEC Alpha (Windows NT)



Memory Operand Example 1

- C code:

`g = h + A[8];`

- `g` in `$s1`, `h` in `$s2`, base address of `A` in `$s3`

- Compiled MIPS code:

- Index 8 requires offset of 32

- 4 bytes per word

```
lw    $t0, 32($s3)    # load word
add   $s1, $s2, $t0
```

offset

base register

Memory Operand Example 2

- C code:

`A[12] = h + A[8];`

- `h` in `$s2`, base address of `A` in `$s3`

- Compiled MIPS code:

- Index 8 requires offset of 32

```
lw    $t0, 32($s3)    # load word
add   $t0, $s2, $t0
sw    $t0, 48($s3)    # store word
```

Registers vs. Memory

- Registers are faster to access than memory
 - Operating on memory data requires loads and stores
 - More instructions to be executed
- **Compilers** use registers for variables as much as possible
 - Only “spill” to memory for less frequently used variables
 - Register optimization is important!

Immediate Operands

- Constant data specified in an instruction

`addi $s3, $s3, 4`

- No subtract immediate instruction

- Just use a negative constant

`addi $s2, $s1, -1`

- *Design Principle 3:* Make the common case fast

- Small constants are common
- Immediate operand avoids a load instruction

Number types

- Integer numbers, unsigned
 - Address calculations
 - Numbers that can only be positive
- Signed numbers
 - Positive
 - Negative
- Floating point numbers
 - numeric calculations
 - Different grades of precision
 - Single precision (IEEE 754)
 - Double precision (IEEE 754)
 - Quadruple precision

Number formats

- Sign and magnitude
- 2's complement
- 1's complement
 - similar to 2's complement, + 0 & - 0
- Biased notation
 - 1000 0000 = minimal negative value (-2^7)
 - 0111 1111 = maximal positive value (2^7-1)
- Representation
 - Binary
 - Decimal
 - Hexadecimal

The Constant Zero

- MIPS register 0 (**\$zero**) is the constant 0
 - Cannot be overwritten
- Useful for common operations
 - E.g., move between registers
`add $t2, $s1, $zero`

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: 0 to $+2^n - 1$

- Example

- $$\begin{aligned} &0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2 \\ &= 0 + \dots + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 0 + \dots + 8 + 0 + 2 + 1 = 11_{10} \end{aligned}$$

- Using 32 bits

- 0 to +4,294,967,295

Signed number representation

- First idea:

Positive and negative numbers

- Take one bit (e.g. 31) as the **sign bit**

- Problem

- **0** 0000000 = 0 positive zero!

- **1** 0000000 = 0 negative zero!

- Each comparison to 0 requires two steps

- 1's complement

- 2's complement

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$

- Example

- $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
 $= -1 \times 2^{31} + 1 \times 2^{30} + \dots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$
 $= -2,147,483,648 + 2,147,483,644 = -4_{10}$

- Using 32 bits

- $-2,147,483,648$ to $+2,147,483,647$

2s-Complement Signed Integers

- Bit 31 is sign bit
 - 1 for negative numbers
 - 0 for non-negative numbers
- $-(-2^n - 1)$ can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
 - 0: 0000 0000 ... 0000
 - -1: 1111 1111 ... 1111
 - Most-negative: 1000 0000 ... 0000
 - Most-positive: 0111 1111 ... 1111
- By default, all data in MIPS is a signed integer
 - add vs. addu

Signed Negation

- Complement and add 1
 - Complement means $1 \rightarrow 0, 0 \rightarrow 1$

$$x + \bar{x} = 1111 \dots 111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
 - $+2 = 0000 \ 0000 \dots 0010_2$
 - $-2 = 1111 \ 1111 \dots 1101_2 + 1$
 $= 1111 \ 1111 \dots 1110_2$

Sign Extension

- How to represent a number using more bits?
 - e.g. 16 bit number as a 32 bit number
 - Must preserve the numeric value
- Replicate the sign bit to the left
 - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
 - +2: 0000 0010 => 0000 0000 0000 0010
 - -2: 1111 1110 => 1111 1111 1111 1110
- In MIPS instruction set
 - addi: extend immediate value
 - lb, lh: extend loaded byte/halfword
 - beq, bne: extend the displacement

Representing Instructions

- Instructions are encoded in binary
 - Called **machine code**
- MIPS instructions
 - Encoded as 32-bit instruction words
 - Small number of formats encoding operation code (opcode), register numbers, ...
 - Regularity!
- Register numbers
 - **\$t0 – \$t7** are reg's 8 – 15
 - **\$t8 – \$t9** are reg's 24 – 25
 - **\$s0 – \$s7** are reg's 16 – 23

MIPS-32 ISA

Registers

❑ Instruction Categories

1. Computational
2. Load/Store
3. Jump and Branch
4. Floating Point
--coprocessor
5. Memory Management
6. Special

R0 - R31

PC

HI

LO

Instruction Formats: all 32 bits wide

op	rs	rt	rd	sa	funct	R format
op	rs	rt	immediate			I format
op	jump target					J format

MIPS R-format Instructions



■ Instruction fields

- **op**: operation code (opcode)
- **rs**: first source register number
- **rt**: second source register number
- **rd**: destination register number
- **shamt**: shift amount (00000 for now)
- **funct**: function code (extends opcode)

R-format Example

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

add \$t0, \$s1, \$s2

special	\$s1	\$s2	\$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$00000010001100100100000000100000_2 = 02324020_{16}$

Hexadecimal

- Base 16
 - Compact representation of bit strings
 - 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

- Example: eca8 6420
 - 1110 1100 1010 1000 0110 0100 0010 0000

MIPS I-format Instructions



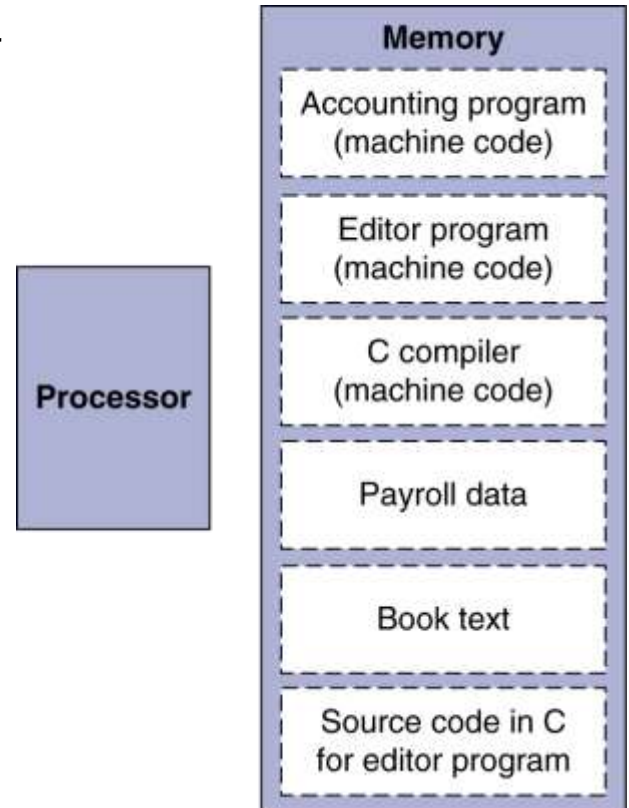
- Immediate arithmetic and load/store instructions
 - **rt**: destination or source register number
 - **Constant**: -2^{15} to $+2^{15} - 1$
 - **Address**: offset added to base address in rs
- **Design Principle 4**: Good design demands good compromises
 - Different formats complicate decoding, but allow 32-bit instructions uniformly
 - Keep formats as similar as possible

Two Key Principles of Computer Design

1. Instructions are represented as numbers and, as such, are indistinguishable from data
2. Programs are stored in alterable memory (that can be read or written to) just like data

The BIG Picture

- ❑ Stored-program concept
 1. Programs can **be shipped as files** of binary numbers – binary compatibility
 2. Computers can inherit ready-made software provided they are compatible with an existing ISA – **leads industry to align around a small number of ISAs**



Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR			or, ori
Bitwise NOT	~	~	nor

- Useful for extracting and inserting groups of bits in a word

Shift Operations

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- **shamt**: how many positions to shift
- Shift left logical
 - Shift left and fill with 0 bits
 - `sll` by i bits multiplies by 2^i
- Shift right logical
 - Shift right and fill with 0 bits
 - `srl` by i bits divides by 2^i (unsigned only)

AND Operations

- Useful to mask bits in a word
 - Select some bits, clear others to 0

and \$t0, \$t1, \$t2

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0000 1100 0000 0000

OR Operations

- Useful to include bits in a word
 - Set some bits to 1, leave others unchanged
- or `$t0`, `$t1`, `$t2`

`$t2` 0000 0000 0000 0000 0000 1101 1100 0000

`$t1` 0000 0000 0000 0000 0011 1100 0000 0000

`$t0` 0000 0000 0000 0000 0011 1101 1100 0000

NOT Operations

- Useful to invert bits in a word
 - Change 0 to 1, and 1 to 0
- MIPS has NOR 3-operand instruction
 - $a \text{ NOR } b == \text{NOT} (a \text{ OR } b)$

`nor $t0, $t1, $zero`

Register 0: always read as zero

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

\$t0 1111 1111 1111 1111 1100 0011 1111 1111

Conditional Operations

- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- `beq rs, rt, L1`
 - if (`rs == rt`) branch to instruction labeled L1;
- `bne rs, rt, L1`
 - if (`rs != rt`) branch to instruction labeled L1;
- `j L1`
 - unconditional jump to instruction labeled L1

Compiling If Statements

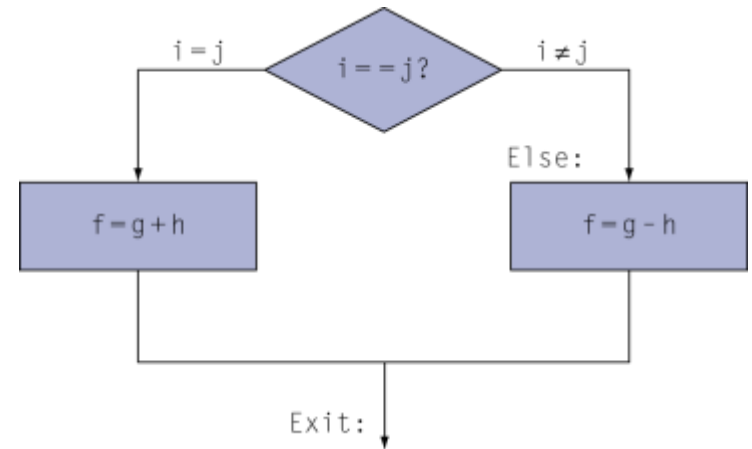
- C code:

```
if (i==j) f = g+h;  
else f = g-h;
```

- f, g, ... in **\$s0, \$s1, ...**

- Compiled MIPS code:

```
        bne $s3, $s4, Else  
        add $s0, $s1, $s2  
        j   Exit  
Else:   sub $s0, $s1, $s2  
Exit:   ...
```



Assembler calculates addresses

Compiling Loop Statements

- C code:

```
while (save[i] == k) i += 1;
```

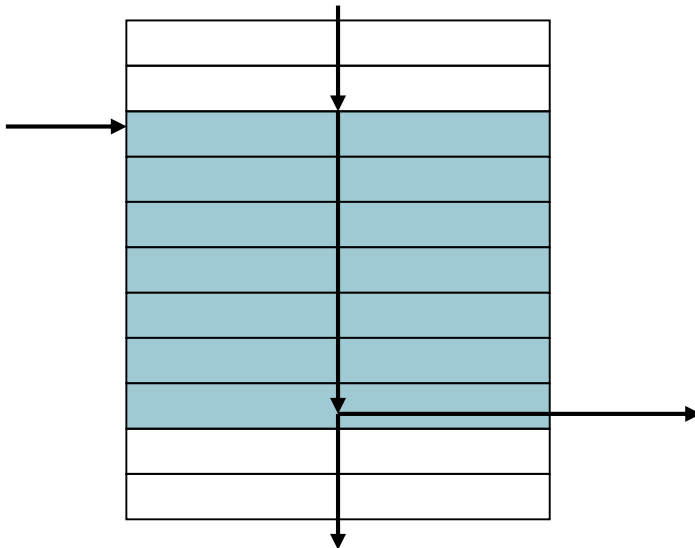
- i in **\$s3**, k in **\$s5**, address of save in **\$s6**

- Compiled MIPS code:

```
Loop:  sll    $t1, $s3, 2    # $t1 = i*4
        add   $t1, $t1, $s6
        lw    $t0, 0($t1)
        bne   $t0, $s5, Exit
        addi  $s3, $s3, 1
        j     Loop
Exit:  ...
```

Basic Blocks

- A basic block is a sequence of instructions with
 - No embedded branches (except at end)
 - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

More Conditional Operations

- Set result to 1 if a condition is true
 - Otherwise, set to 0
- `slt rd, rs, rt`
 - if ($rs < rt$) $rd = 1$; else $rd = 0$;
- `slti rt, rs, constant`
 - if ($rs < \text{constant}$) $rt = 1$; else $rt = 0$;
- Use in combination with `beq`, `bne`

```
    slt $t0, $s1, $s2    # if ($s1 < $s2)
    bne $t0, $zero, L    #   branch to L
```

Branch Instruction Design

- Why not **b1t**, **bge**, etc?
- Hardware for $<$, $\geq$, ... slower than $=$, $\neq$
 - Combining with branch involves more work per instruction, requiring a slower clock
 - All instructions penalized!
- **beq** and **bne** are the common case
- This is a good design compromise

Signed vs. Unsigned

- Signed comparison: `slt, slti`
- Unsigned comparison: `sltu, sltui`
- Example
 - `$s0` = 1111 1111 1111 1111 1111 1111 1111 1111
 - `$s1` = 0000 0000 0000 0000 0000 0000 0000 0001
 - `slt $t0, $s0, $s1 # signed`
 - $-1 < +1 \Rightarrow \$t0 = 1$
 - `sltu $t0, $s0, $s1 # unsigned`
 - $+4,294,967,295 > +1 \Rightarrow \$t0 = 0$

Procedure Calling

- Procedures enable **structured** programs
 - Easier to understand and reuse code
- Steps required
 1. Place parameters in registers
 2. Transfer control to procedure
 3. Acquire storage for procedure
 4. Perform procedure's operations
 5. Place result in register for caller
 6. Return to place of call

Register Usage

- **\$a0 – \$a3**: arguments (reg's 4 – 7)
- **\$v0, \$v1**: result (i.e. return) values (reg's 2 and 3)
- **\$t0 – \$t9**: temporaries
 - Can be overwritten by callee
- **\$s0 – \$s7**: saved
 - Must be saved/restored by callee
- **\$gp**: global pointer for static data (reg 28)
- **\$sp**: stack pointer (reg 29)
- **\$fp**: frame pointer (reg 30)
- **\$ra**: return address (reg 31)

Procedure Call Instructions

- Procedure call: jump and link
`jal ProcedureLabel`
 - Address of following instruction put in `$ra`
 - Jumps to target address
- Procedure return: jump register
`jr $ra`
 - Copies `$ra` to program counter
 - Can also be used for computed jumps
 - e.g., for case/switch statements

Leaf Procedure Example

- C code:

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Arguments g, ..., j in **\$a0, ..., \$a3**
- f in **\$s0** (hence, need to save **\$s0** on stack)
- Result in **\$v0**

Leaf Procedure Example

- MIPS code:

leaf_example:			
addi	\$sp,	\$sp, -4	Save \$s0 on stack
sw	\$s0,	0(\$sp)	
add	\$t0,	\$a0, \$a1	Procedure body
add	\$t1,	\$a2, \$a3	
sub	\$s0,	\$t0, \$t1	
add	\$v0,	\$s0, \$zero	Result
lw	\$s0,	0(\$sp)	Restore \$s0
addi	\$sp,	\$sp, 4	
jr	\$ra		Return

Think about the differences between 80x86 and MIPS Procedure Instructions!

Non-Leaf Procedures

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
 - Any saved registers being used \$s0-\$s7
 - Its return address
 - Any arguments and temporaries needed after the call
 - e.g., suppose main program calls procedure A with an argument 3 in \$a0 and then using **jal A**
 - Then if procedure A calls procedure B via **jal B** with an argument of 7 placed in \$a0, there is a conflict over the use of register \$a0
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
int fact (int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

- Argument n in **\$a0**
- Result in **\$v0**

Non-Leaf Procedure Example

- MIPS code:

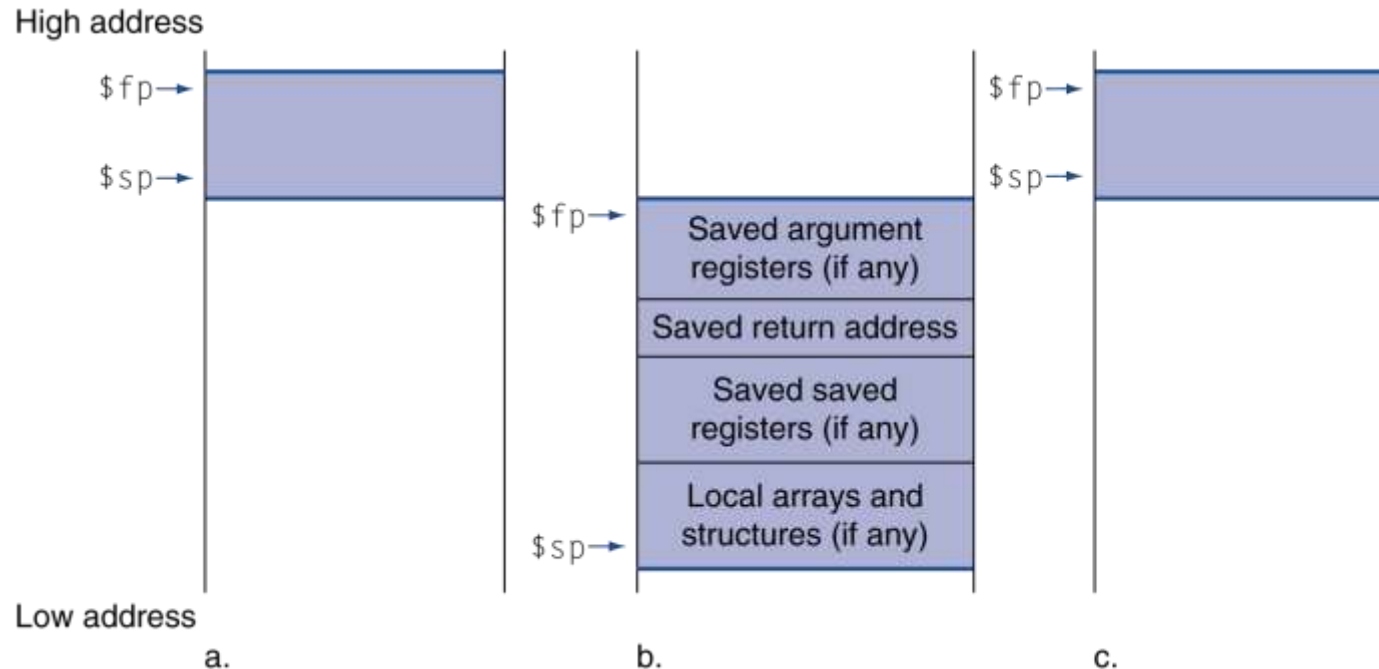
fact:		
addi	\$sp, \$sp, -8	# adjust stack for 2 items
sw	\$ra, 4(\$sp)	# save return address
sw	\$a0, 0(\$sp)	# save argument
slti	\$t0, \$a0, 1	# test for n < 1
beq	\$t0, \$zero, L1	
addi	\$v0, \$zero, 1	# if so, result is 1
addi	\$sp, \$sp, 8	# pop 2 items from stack
jr	\$ra	# and return
L1:	addi \$a0, \$a0, -1	# else decrement n
	jal fact	# recursive call
lw	\$a0, 0(\$sp)	# restore original n
lw	\$ra, 4(\$sp)	# and return address
addi	\$sp, \$sp, 8	# pop 2 items from stack
mul	\$v0, \$a0, \$v0	# multiply to get result
jr	\$ra	# and return

MIPS Register Conventions

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

Register 1, called **\$at**, is reserved for the assembler and registers 26-27, called **\$k0-\$k1**, are reserved for the operating system.

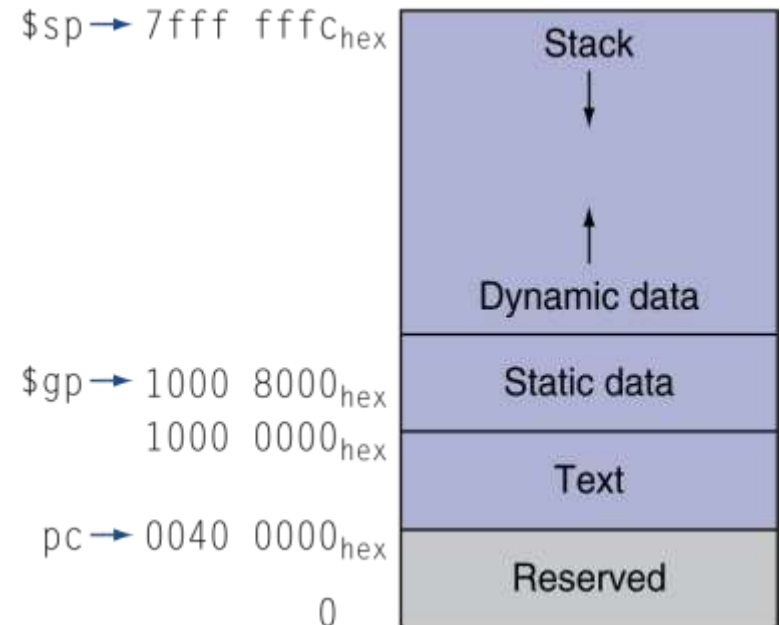
Local Data on the Stack



- Local data allocated by callee
 - e.g., C automatic variables
- Procedure frame (activation record)
 - Used by some compilers to manage stack storage

Memory Layout

- **Text:** program code
- **Static data:** global variables
 - e.g., static variables in C, constant arrays and strings
 - \$gp initialized to address allowing $\pm$ offsets into this segment
- **Dynamic data:** heap
 - E.g., malloc() in C, new in Java, linked lists
 - Must use free() in C
 - Memory leak issues!
- **Stack:** automatic storage



Character Data

- Byte-encoded character sets
 - **ASCII**: 128 characters
 - 95 graphic, 33 non-printable control (almost obsolete now)

ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

- **Latin-1**: 256 characters
 - ASCII, +96 more graphic characters

Character Data

- **Unicode:** 32-bit character set
 - Used in Java, C++ wide characters, ...
 - UTF-8, UTF-16: variable-length encodings
 - Most of the world's alphabets, plus symbols

Latin	Malayalam	Tagbanwa	General Punctuation
Greek	Sinhala	Khmer	Spacing Modifier Letters
Cyrillic	Thai	Mongolian	Currency Symbols
Armenian	Lao	Limbu	Combining Diacritical Marks
Hebrew	Tibetan	Tai Le	Combining Marks for Symbols
Arabic	Myanmar	Kangxi Radicals	Superscripts and Subscripts
Syriac	Georgian	Hiragana	Number Forms
Thaana	Hangul Jamo	Katakana	Mathematical Operators
Devanagari	Ethiopic	Bopomofo	Mathematical Alphanumeric Symbols
Bengali	Cherokee	Kanbun	Braille Patterns
Gurmukhi	Unified Canadian Aboriginal Syllabic	Shavian	Optical Character Recognition
Gujarati	Ogham	Osmanya	Byzantine Musical Symbols
Oriya	Runic	Cypriot Syllabary	Musical Symbols
Tamil	Tagalog	Tai Xuan Jing Symbols	Arrows
Telugu	Hanunoo	Yijing Hexagram Symbols	Box Drawing
Kannada	Buhid	Aegean Numbers	Geometric Shapes

Byte/Halfword Operations

- To manipulate text (e.g. ASCII characters) byte operations are essential
- MIPS byte/halfword load/store

- String processing is a common case

`lb rt, offset(rs)` `lh rt, offset(rs)`

- Sign extend to 32 bits in rt

`lbu rt, offset(rs)` `lhu rt, offset(rs)`

- Zero extend to 32 bits in rt

`sb rt, offset(rs)` `sh rt, offset(rs)`

- Store just rightmost byte/halfword

String Copy Example

- C code (naïve):

- Null-terminated string

```
void strcpy (char x[], char y[])  
{ int i;  
  i = 0;  
  while ((x[i]=y[i])!='\0')  
    i += 1;  
}
```

- Addresses of x, y in **\$a0, \$a1**
- i in **\$s0**

String Copy Example

- MIPS code:

strcpy:		
	addi \$sp, \$sp, -4	# adjust stack for 1 item
	sw \$s0, 0(\$sp)	# save \$s0
	add \$s0, \$zero, \$zero	# i = 0
L1:	add \$t1, \$s0, \$a1	# addr of y[i] in \$t1
	lbu \$t2, 0(\$t1)	# \$t2 = y[i]
	add \$t3, \$s0, \$a0	# addr of x[i] in \$t3
	sb \$t2, 0(\$t3)	# x[i] = y[i]
	beq \$t2, \$zero, L2	# exit loop if y[i] == 0
	addi \$s0, \$s0, 1	# i = i + 1
	j L1	# next iteration of loop
L2:	lw \$s0, 0(\$sp)	# restore saved \$s0
	addi \$sp, \$sp, 4	# pop 1 item from stack
	jr \$ra	# and return

32-bit Constants

- Most constants are small
 - 16-bit immediate is sufficient
- For the occasional 32-bit constant
 - `lui rt, constant`
 - Copies 16-bit const to left 16 bits of rt; Clears right 16 bits of rt
- Example: how can we load 32 bit constant into `$s0`?

0000 0000 0111 1101 0000 1001 0000 0000

`lui $s0, 61`

0000 0000 0111 1101	0000 0000 0000 0000
---------------------	---------------------

`ori $s0, $s0, 2304`

0000 0000 0111 1101	0000 1001 0000 0000
---------------------	---------------------

- Typically **assembler** is responsible for breaking a large constant and reassembling into a register (using the reserved register `$at`)

Branch Addressing

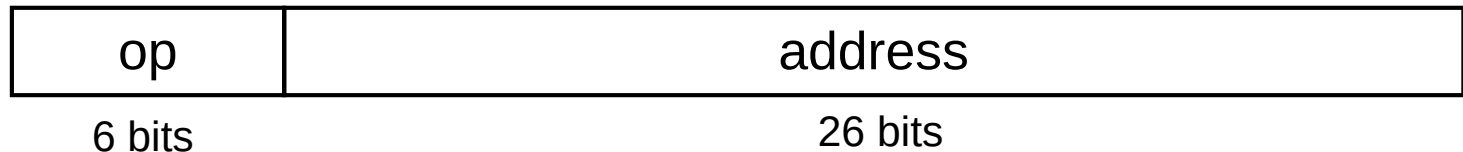
- Branch instructions specify
 - Opcode, two registers, target address
- If addresses had to fit in 16-bits, then no program could be bigger than 2^{16}



- PC-relative addressing
 - Target address = PC + offset $\times$ 4
 - PC already incremented by 4 by this time
 - Almost all loops and *if* statements $< 2^{16}$ words

Jump Addressing

- Jump (**j** and **jal**) targets could be anywhere in text segment
 - Encode full address in instruction



- (Pseudo)Direct jump addressing
 - Target address = $PC_{31..28} : (\text{address} \times 4)$

Branching Far Away

- If branch target is too far to encode with 16-bit offset, assembler rewrites the code
- Example: how can we increase branching distance for the following instruction?

```
beq $s0,$s1, L1
```



```
bne $s0,$s1, L2
```

```
j L1
```

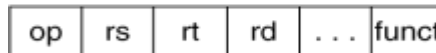
```
L2: ...
```

Addressing Mode Summary

1. Immediate addressing



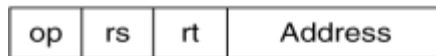
2. Register addressing



Registers

Register

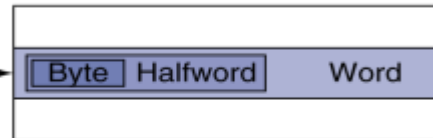
3. Base addressing



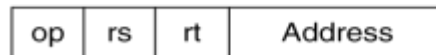
Memory



+



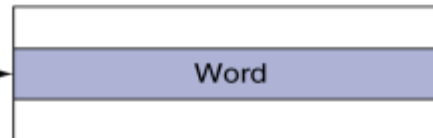
4. PC-relative addressing



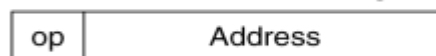
Memory



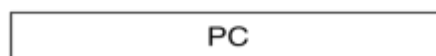
+



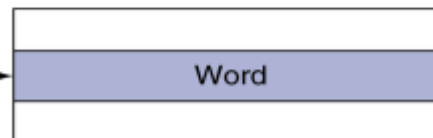
5. Pseudodirect addressing



Memory



+



Synchronization

- Two processors sharing an area of memory
 - P1 writes, then P2 reads
 - Data race if P1 and P2 don't synchronize
 - Result depends of order of accesses
- Hardware and ISA support required
 - Atomic read/write memory operation
 - No other access to the location allowed between the read and write
- Could be a single instruction
 - E.g., atomic swap of register $\leftrightarrow$ memory
 - Or an atomic pair of instructions

Motivation for Atomic Swap

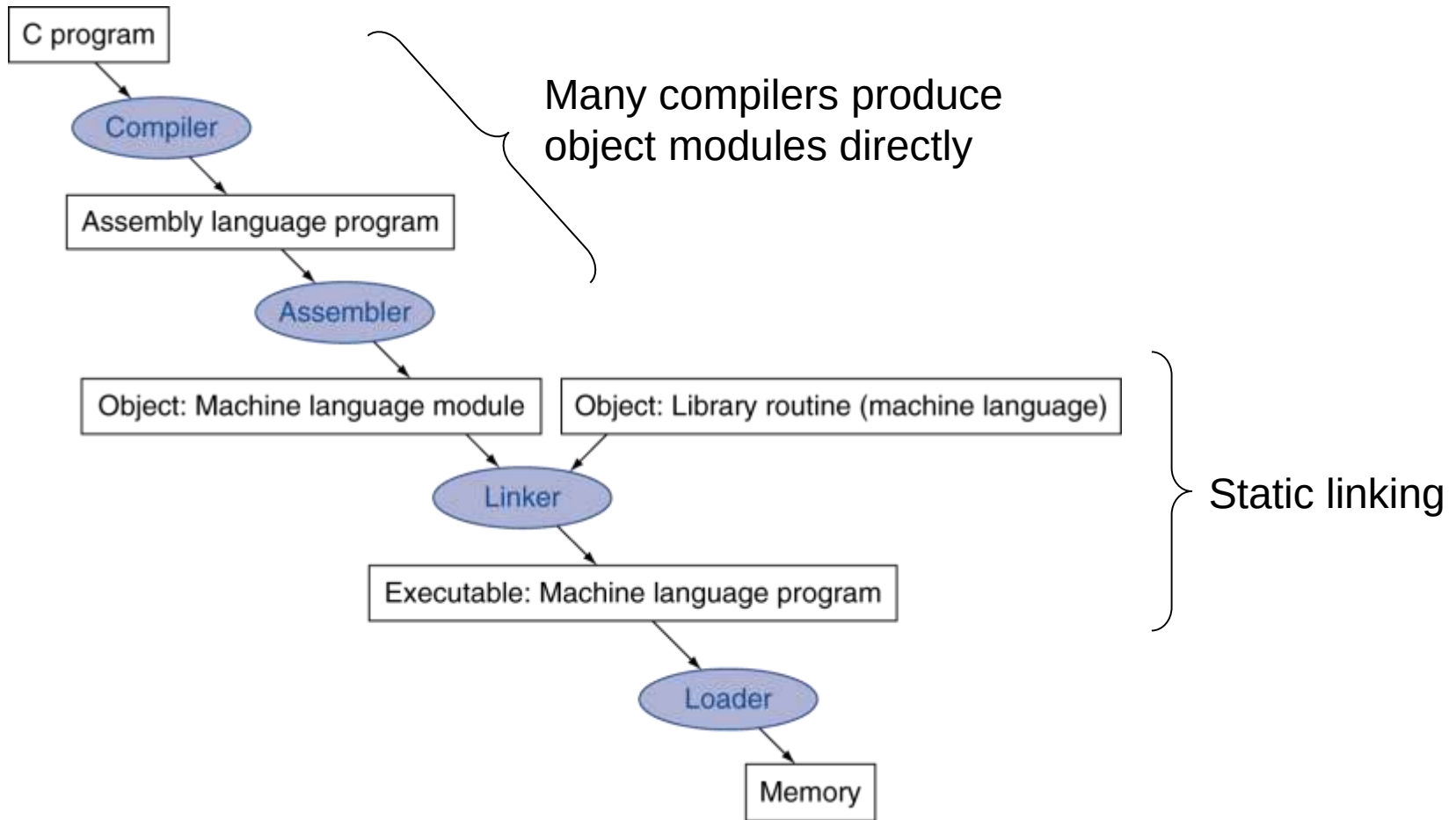
- Build lock for multi-processor system
 - To govern access to some critical resource
 - 0 indicates lock is free; 1 indicates lock is unavailable; “lock” stored in memory
 - Processor tries to set the lock by doing an exchange of 1 which is in a register
 - Value returned from exchange is
 - 1 if another processor already acquired the lock;
 - 0 otherwise; in this case value changed to 1 to prevent another processor from retrieving 0
 - Locks allow breaking race conditions

Synchronization in MIPS

- Load linked: `ll rt, offset(rs)`
- Store conditional: `sc rt, offset(rs); rt->(rs)`
 - Succeeds if location not changed since the `ll`
 - Returns 1 in `rt`
 - Fails if location is changed
 - Returns 0 in `rt`
- Example: atomic swap (to test/set lock variable)

```
try: add $t0,$zero,$s4 ;copy exchange value
      ll  $t1,0($s1)    ;load linked 相当于加锁机制
      sc  $t0,0($s1)    ;store conditional
      beq $t0,$zero,try ;branch store fails
      add $s4,$zero,$t1 ;put load value in $s4
```
- `ll`, `sc` can be used to build other synchronization primitives: atomic compare and swap, fetch and increment, etc.

Translation and Startup



Assembler Pseudoinstructions

- Most assembler instructions represent machine instructions one-to-one
- **Pseudoinstructions**: figments of the assembler's imagination

`move $t0, $t1` $\rightarrow$ `add $t0, $zero, $t1`

`blt $t0, $t1, L` $\rightarrow$ `slt $at, $t0, $t1`
 `bne $at, $zero, L`

- **\$at** (register 1): assembler temporary

Producing an Object Module

- Assembler (or compiler) translates program into **machine** instructions
- Provides information for building a complete program from the pieces
- E.g. object file (*.o) for UNIX has 6 distinct parts:
 - **Header**: described contents of object module
 - **Text segment**: translated instructions
 - **Static data segment**: data allocated for the life of the program
 - **Relocation info**: for contents that depend on absolute location of loaded program
 - **Symbol table**: global definitions and external refs
 - **Debug info**: for associating machine code with high level source code by debuggers

Linking Object Modules

- Produces an executable image
 1. Merges segments/object files/procedures
 - Rather than compiling/assembling whole program when a single line of a procedure is changed, procedures are compiled/assembled independently
 2. Resolve labels (determine their addresses)
 3. Patch location-dependent and external refs

Loading a Program

- Load from image file on disk into memory
 1. Read header to determine segment sizes
 2. Create virtual address space
 3. Copy text and initialized data into memory
 - Or set page table entries so they can be faulted in
 4. Set up arguments on stack
 5. Initialize registers (including `$sp`, `$fp`, `$gp`)
 6. Jump to startup routine
 - Copies arguments to `$a0`, ... and calls `main`
 - When `main` returns, do `exit` syscall

Static and Dynamic Linking

- So far discussed traditional approach to linking libraries before program executes
 - **Static linking**
 - If new version of a library released to fix bugs or support new hardware, statically linked program keeps using old version
 - All routines in a library are loaded, even if they are not used
- **Dynamic linking**: Do not link/load library routines until program is run
 - Requires procedure code to be relocatable
 - Automatically picks up new library versions
 - Dynamically linked libraries (DLLs)

Lazy Linkage

Routine linked only **after** it is called

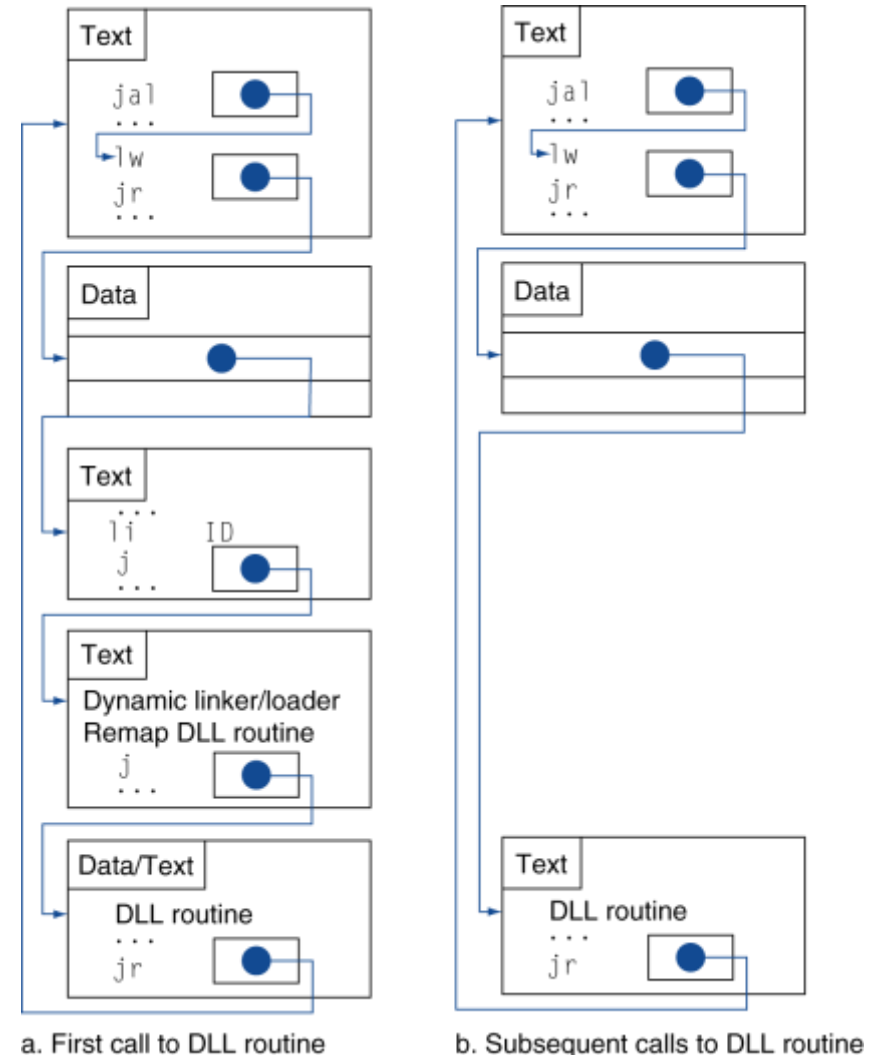
Avoids image bloat caused by static linking of all referenced libraries

Indirection table

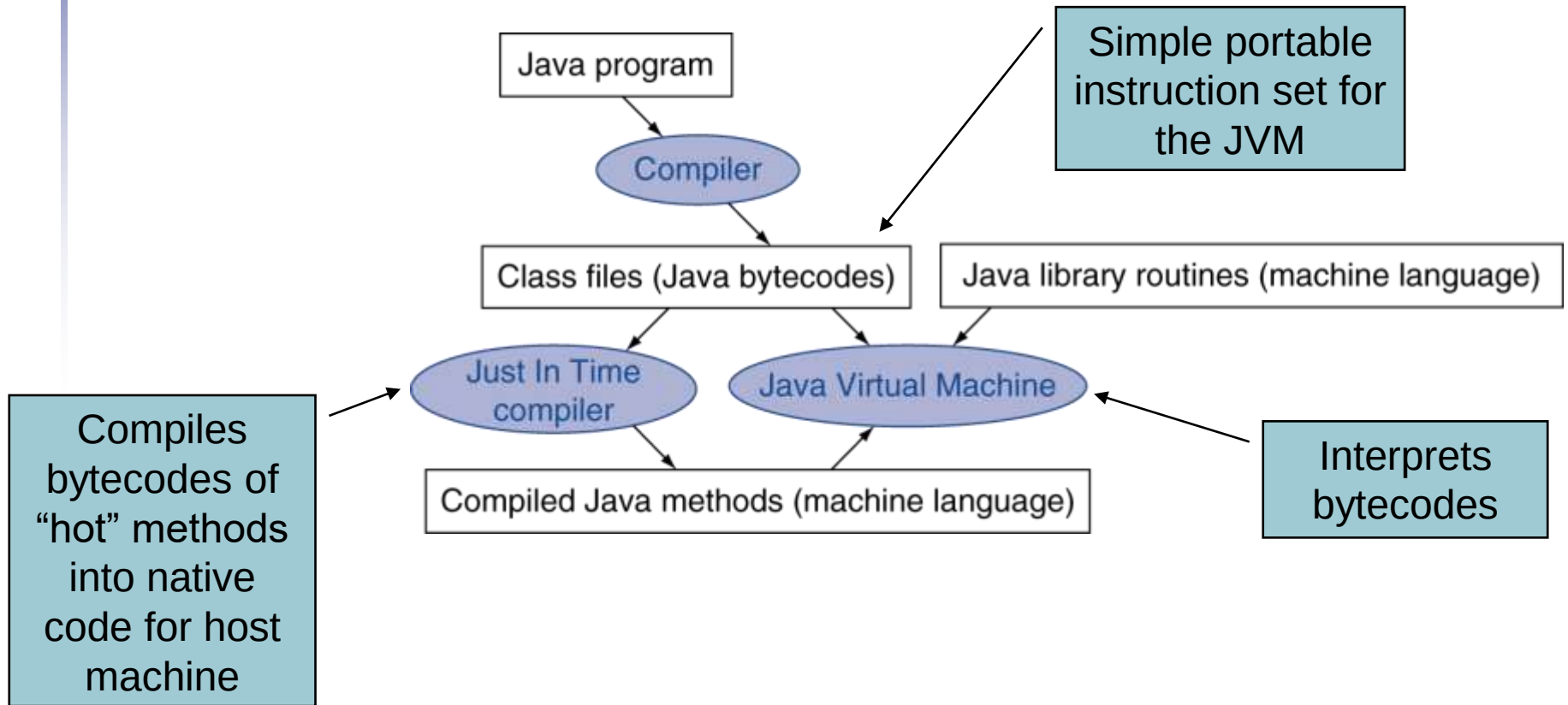
Stub: Loads routine ID,
Jump to linker/loader

Linker/loader code

Dynamically
mapped code



Starting Java Applications



C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

- v in **\$a0**, k in **\$a1**, temp in **\$t0**

The Procedure Swap

swap: sll \$t1, \$a1, 2	# \$t1 = k * 4
add \$t1, \$a0, \$t1	# \$t1 = v+(k*4)
	# (address of v[k])
lw \$t0, 0(\$t1)	# \$t0 (temp) = v[k]
lw \$t2, 4(\$t1)	# \$t2 = v[k+1]
sw \$t2, 0(\$t1)	# v[k] = \$t2 (v[k+1])
sw \$t0, 4(\$t1)	# v[k+1] = \$t0 (temp)
jr \$ra	# return to calling routine

- Forgetting that sequential word addresses **differ by 4 instead of 1** is a common mistake in assembly language programming

The Sort Procedure in C

- Non-leaf bubble/exchange sort (calls swap)

```
void sort (int v[], int n)
{
    int i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
            swap(v, j);
        }
    }
}
```

- v in **\$a0**, n in **\$a1**, i in **\$s0**, j in **\$s1**

The Procedure Body

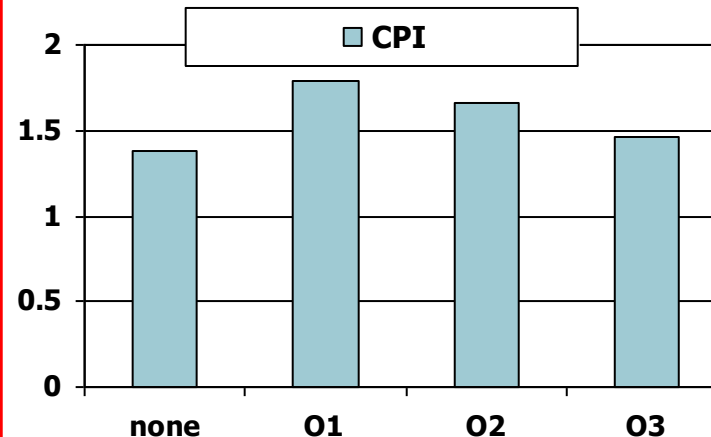
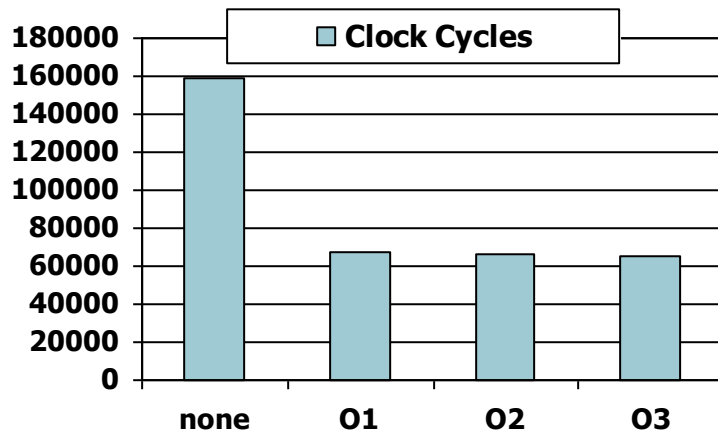
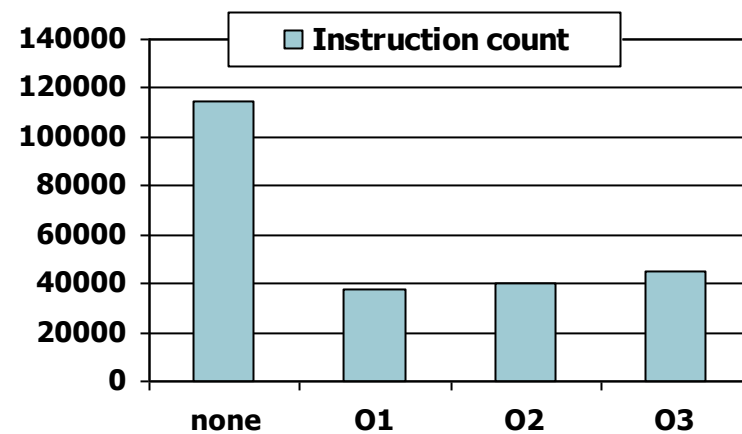
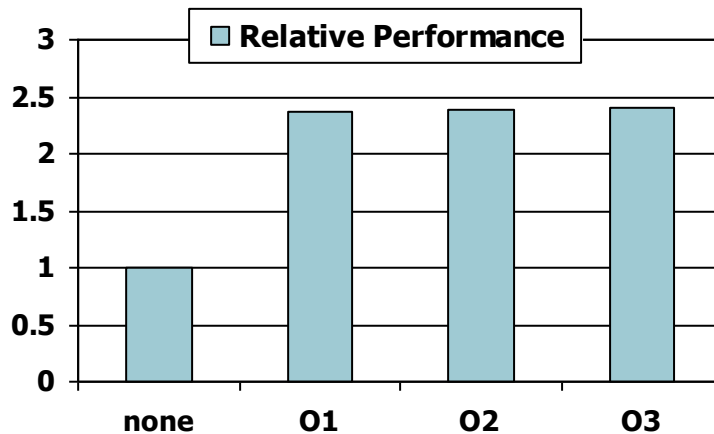
	move \$s2, \$a0	# save \$a0 into \$s2	Move params
	move \$s3, \$a1	# save \$a1 into \$s3	
	move \$s0, \$zero	# i = 0	
for1tst:	slt \$t0, \$s0, \$s3	# \$t0 = 0 if \$s0 ≥ \$s3 (i ≥ n)	Outer loop
	beq \$t0, \$zero, exit1	# go to exit1 if \$s0 ≥ \$s3 (i ≥ n)	
	addi \$s1, \$s0, -1	# j = i - 1	
for2tst:	slti \$t0, \$s1, 0	# \$t0 = 1 if \$s1 < 0 (j < 0)	
	bne \$t0, \$zero, exit2	# go to exit2 if \$s1 < 0 (j < 0)	
	sll \$t1, \$s1, 2	# \$t1 = j * 4	Inner loop
	add \$t2, \$s2, \$t1	# \$t2 = v + (j * 4)	
	lw \$t3, 0(\$t2)	# \$t3 = v[j]	
	lw \$t4, 4(\$t2)	# \$t4 = v[j + 1]	
	slt \$t0, \$t4, \$t3	# \$t0 = 0 if \$t4 ≥ \$t3	
	beq \$t0, \$zero, exit2	# go to exit2 if \$t4 ≥ \$t3	
	move \$a0, \$s2	# 1st param of swap is v (old \$a0)	Pass params & call
	move \$a1, \$s1	# 2nd param of swap is j	
	jal swap	# call swap procedure	
	addi \$s1, \$s1, -1	# j -= 1	Inner loop
	j for2tst	# jump to test of inner loop	
exit2:	addi \$s0, \$s0, 1	# i += 1	
	j for1tst	# jump to test of outer loop	Outer loop

The Full Procedure

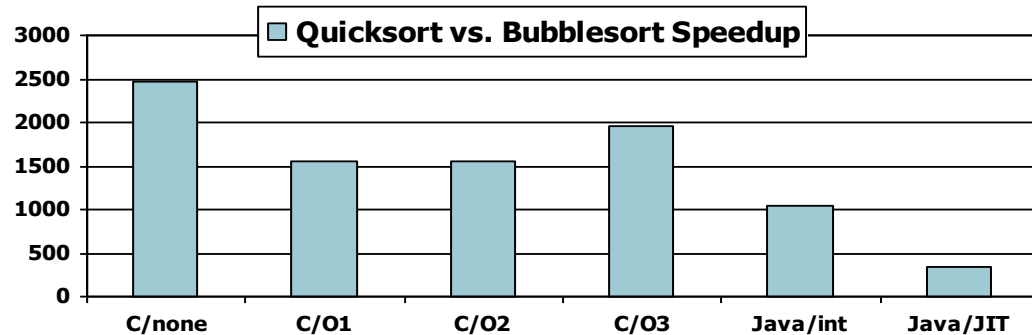
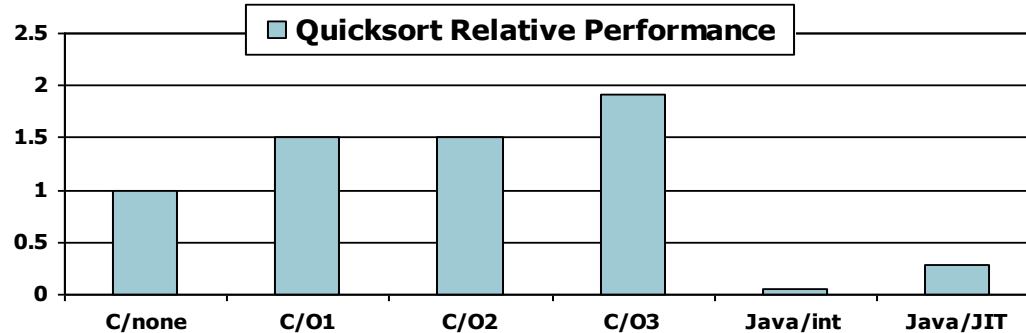
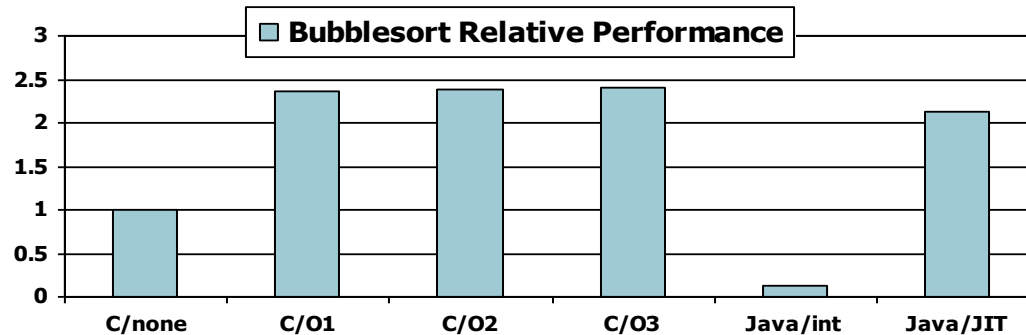
sort:	addi \$sp,\$sp, -20	# make room on stack for 5 registers
	sw \$ra, 16(\$sp)	# save \$ra on stack
	sw \$s3,12(\$sp)	# save \$s3 on stack
	sw \$s2, 8(\$sp)	# save \$s2 on stack
	sw \$s1, 4(\$sp)	# save \$s1 on stack
	sw \$s0, 0(\$sp)	# save \$s0 on stack
	...	# procedure body
	...	
	exit1: lw \$s0, 0(\$sp)	# restore \$s0 from stack
	lw \$s1, 4(\$sp)	# restore \$s1 from stack
	lw \$s2, 8(\$sp)	# restore \$s2 from stack
	lw \$s3,12(\$sp)	# restore \$s3 from stack
	lw \$ra,16(\$sp)	# restore \$ra from stack
	addi \$sp,\$sp, 20	# restore stack pointer
	jr \$ra	# return to calling routine

Effect of Compiler Optimization

Compiled with gcc for Pentium 4 under Linux



Effect of Language and Algorithm



Lessons Learnt

- **Instruction count** and **CPI** are not good performance indicators in isolation
- Compiler optimizations are sensitive to the algorithm
- Java/JIT compiled code is significantly faster than JVM interpreted
 - Comparable to optimized C in some cases
- Nothing can fix a dumb algorithm!

Arrays vs. Pointers

- Array indexing involves
 - Multiplying index by element size
 - Adding to array base address
- Pointers correspond directly to memory addresses
 - Can avoid indexing complexity

Example: Clearing an Array

```
clear1(int array[], int size) {  
    int i;  
    for (i = 0; i < size; i += 1)  
        array[i] = 0;  
}
```

```
        move $t0,$zero    # i = 0  
loop1: sll $t1,$t0,2      # $t1 = i * 4  
        add $t2,$a0,$t1  # $t2 =  
                        # &array[i]  
        sw $zero, 0($t2) # array[i] = 0  
        addi $t0,$t0,1   # i = i + 1  
        slt $t3,$t0,$a1  # $t3 =  
                        # (i < size)  
        bne $t3,$zero,loop1 # if (...)  
                        # goto loop1
```

```
clear2(int *array, int size) {  
    int *p;  
    for (p = &array[0]; p < &array[size];  
        p = p + 1)  
        *p = 0;  
}
```

```
        move $t0,$a0      # p = & array[0]  
        sll $t1,$a1,2     # $t1 = size * 4  
        add $t2,$a0,$t1  # $t2 =  
                        # &array[size]  
loop2: sw $zero,0($t0)    # Memory[p] = 0  
        addi $t0,$t0,4    # p = p + 4  
        slt $t3,$t0,$t2  # $t3 =  
                        # (p < &array[size])  
        bne $t3,$zero,loop2 # if (...)  
                        # goto loop2
```

Comparison of Array vs. Ptr

- Multiply “strength reduced” to shift
- Array version requires shift to be inside loop
 - Part of index calculation for incremented i
 - c.f. incrementing pointer
- **Compiler can achieve same** effect as manual use of pointers
 - Induction variable elimination
 - Better to make program clearer and safer
 - **Avoid using pointers!!**

ARM & MIPS Similarities

- ARM: the most popular embedded core
- Similar basic set of instructions to MIPS

	ARM	MIPS
Date announced	1985	1985
Instruction size	32 bits	32 bits
Address space	32-bit flat	32-bit flat
Data alignment	Aligned	Aligned
Data addressing modes	9	3
Registers	15 × 32-bit	31 × 32-bit
Input/output	Memory mapped	Memory mapped

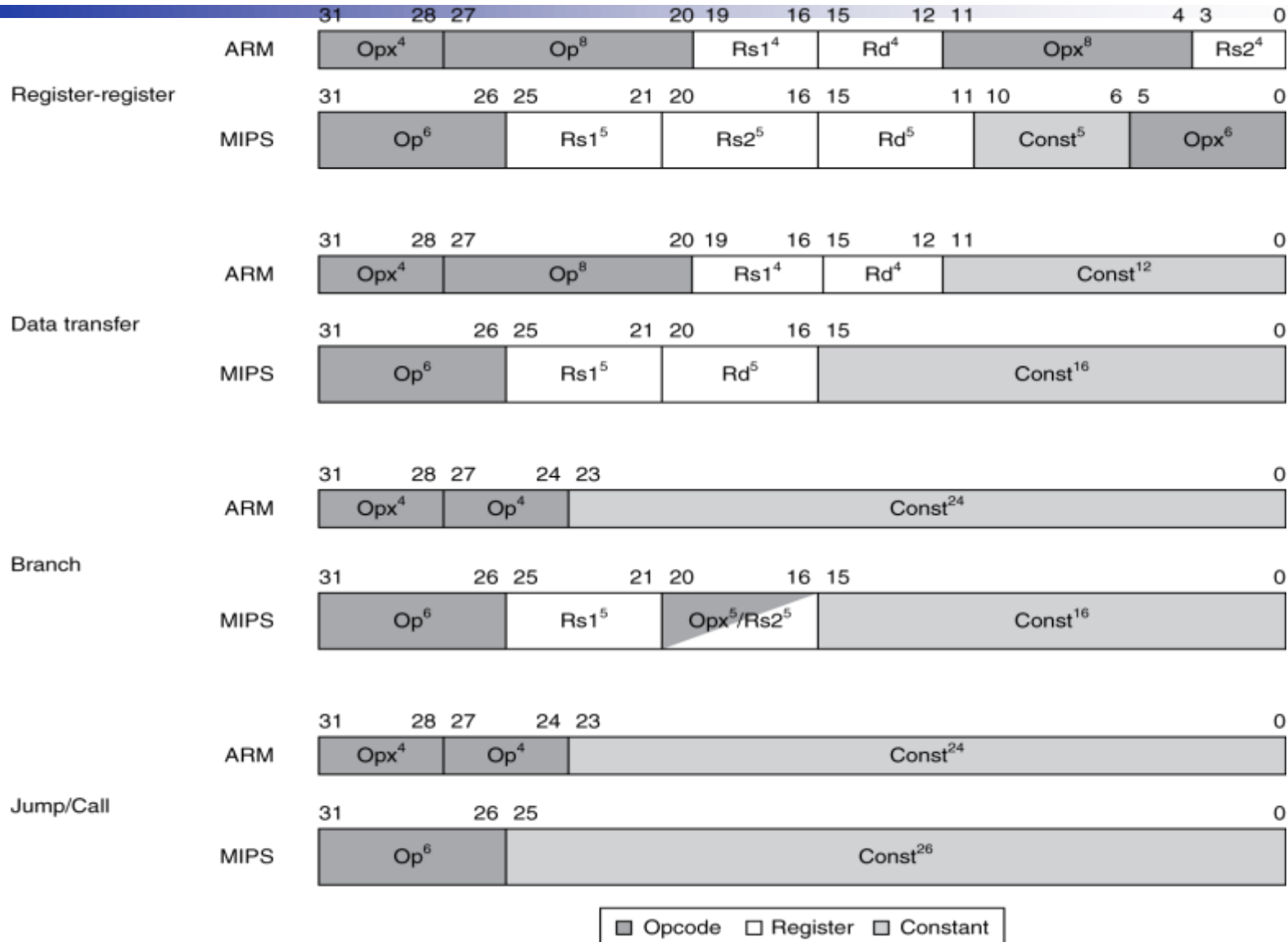
ARM & MIPS Similarities

	Instruction name	ARM	MIPS
Register-register	Add	add	addu, addiu
	Add (trap if overflow)	adds; swivs	add
	Subtract	sub	subu
	Subtract (trap if overflow)	subs; swivs	sub
	Multiply	mul	mult, multu
	Divide	—	div, divu
	And	and	and
	Or	orr	or
	Xor	eor	xor
	Load high part register	—	lui
	Shift left logical	lsl ¹	sllv, sll
	Shift right logical	lsr ¹	srlv, srl
	Shift right arithmetic	asr ¹	srav, sra
	Compare	cmp, cmn, tst, teq	slt/i, slt/iu
Data transfer	Load byte signed	ldrsb	lb
	Load byte unsigned	ldrb	lbu
	Load halfword signed	ldrsh	lh
	Load halfword unsigned	ldrh	lhu
	Load word	ldr	lw
	Store byte	strb	sb
	Store halfword	strh	sh
	Store word	str	sw
	Read, write special registers	mrs, msr	move
	Atomic Exchange	swp, swpb	ll;sc

Compare and Branch in ARM

- Uses condition codes for result of an arithmetic/logical instruction
 - Negative, zero, carry, overflow
 - **Compare (CMP)** instructions to set condition codes without keeping the result
 - Subtracts one operand from the other
- Each instruction can be conditional
 - Top 4 bits of instruction word: condition value
 - Can avoid branches over single instructions

Instruction Encoding



Unique ARM Instructions

Name	Definition	ARM v.4	MIPS
Load immediate	$Rd = Imm$	mov	addi, \$0,
Not	$Rd = \sim(Rs1)$	mvn	nor, \$0,
Move	$Rd = Rs1$	mov	or, \$0,
Rotate right	$Rd = Rs \ll i$ $Rd_{0 \dots i-1} = Rs_{31-i \dots 31}$	ror	
And not	$Rd = Rs1 \& \sim(Rs2)$	bic	
Reverse subtract	$Rd = Rs2 - Rs1$	rsb, rsc	
Support for multiword integer add	CarryOut, $Rd = Rd + Rs1 + OldCarryOut$	adcs	—
Support for multiword integer sub	CarryOut, $Rd = Rd - Rs1 + OldCarryOut$	sbc	—

The Intel x86 ISA

- Evolution with backward compatibility
 - **8080 (1974)**: 8-bit microprocessor
 - Accumulator, plus 3 index-register pairs
 - **8086 (1978)**: 16-bit extension to 8080
 - Complex instruction set (CISC)
 - **8087 (1980)**: floating-point coprocessor
 - Adds FP instructions and register stack
 - **80286 (1982)**: 24-bit addresses, MMU
 - Segmented memory mapping and protection
 - **80386 (1985)**: 32-bit extension (now IA-32)
 - Additional addressing modes and operations
 - Paged memory mapping as well as segments

The Intel x86 ISA

- Further evolution...
 - **i486 (1989)**: pipelined, on-chip caches and FPU
 - Compatible competitors: AMD, Cyrix, ...
 - **Pentium (1993)**: superscalar, 64-bit datapath
 - Later versions added MMX (Multi-Media eXtension) instructions
 - The infamous \$500M FDIV bug
 - **Pentium Pro (1995), Pentium II (1997)**
 - New microarchitecture (see Colwell, *The Pentium Chronicles*)
 - **Pentium III (1999)**
 - Added SSE (Streaming SIMD Extensions) and associated registers
 - **Pentium 4 (2001)**
 - New microarchitecture
 - Added 144 SSE2 instructions

The Intel x86 ISA

- And further...
 - **AMD64 (2003):** extended architecture to 64 bits
 - **EM64T** – Extended Memory 64 Technology (2004)
 - AMD64 adopted by Intel (with refinements)
 - Added SSE3 instructions
 - **Intel Core (2006)**
 - Added SSE4 instructions, virtual machine support
 - **AMD64 (announced 2007):** SSE5 instructions
 - Intel declined to follow, instead...
 - **Advanced Vector Extension (announced 2008)**
 - Longer SSE registers, more instructions
- Existing x86 software base at each step too important to jeopardize with significant architectural changes
 - x86 extended by **one instruction per month** since inception!
 - is an architecture that is difficult to explain and hard to love
 - despite lacking “technical elegance”, x86 family pervasive today

Basic x86 Registers (80386)

Name	31	0	Use
EAX			GPR 0
ECX			GPR 1
EDX			GPR 2
EBX			GPR 3
ESP			GPR 4
EBP			GPR 5
ESI			GPR 6
EDI			GPR 7
	CS		Code segment pointer
	SS		Stack segment pointer (top of stack)
	DS		Data segment pointer 0
	ES		Data segment pointer 1
	FS		Data segment pointer 2
	GS		Data segment pointer 3
EIP			Instruction pointer (PC)
EFLAGS			Condition codes

Basic x86 Addressing Modes

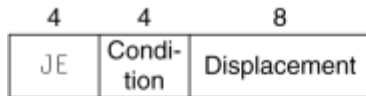
- Two operands per instruction

Source/dest operand	Second source operand
Register	Register
Register	Immediate
Register	Memory
Memory	Register
Memory	Immediate

- Memory addressing modes
 - Address in register
 - $\text{Address} = R_{\text{base}} + \text{displacement}$
 - $\text{Address} = R_{\text{base}} + 2^{\text{scale}} \times R_{\text{index}}$ (scale = 0, 1, 2, or 3)
 - $\text{Address} = R_{\text{base}} + 2^{\text{scale}} \times R_{\text{index}} + \text{displacement}$

x86 Instruction Encoding

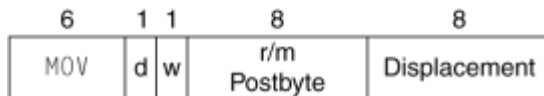
a. JE EIP + displacement



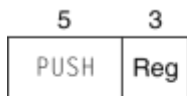
b. CALL



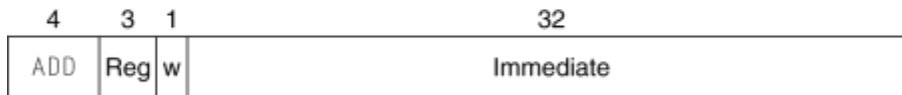
c. MOV EBX, [EDI + 45]



d. PUSH ESI



e. ADD EAX, #6765



f. TEST EDX, #42



■ Variable length encoding

- 1 to 17 bytes
- Rightly called **CISC**
- Postfix bytes specify addressing mode
- Prefix bytes modify operation
 - Operand length, repetition, locking, ...

Implementing IA-32

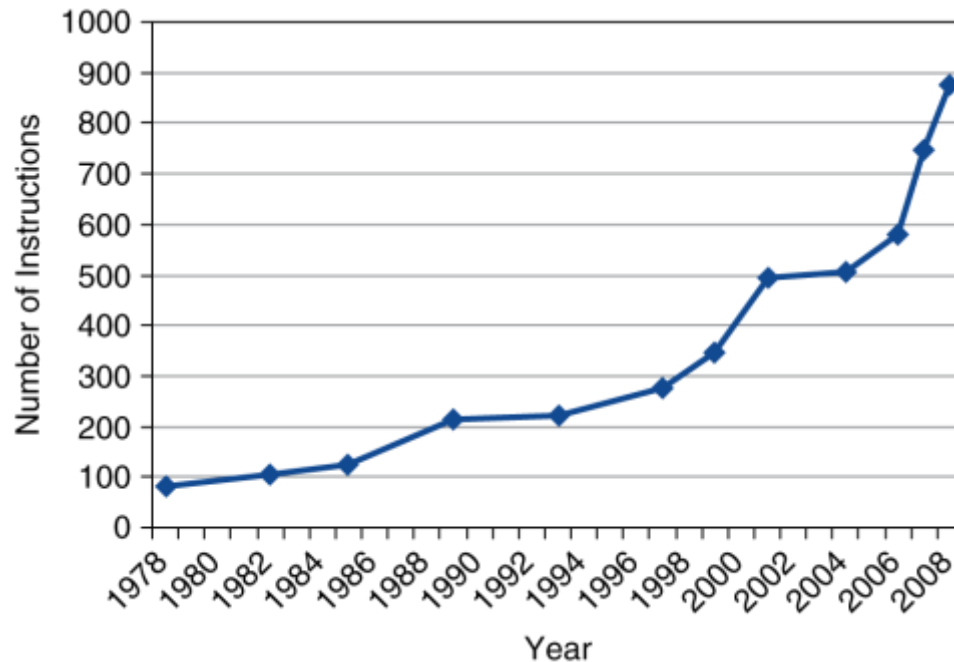
- Complex instruction set makes implementation difficult
 - Hardware translates instructions to simpler micro-operations
 - Simple instructions: 1–1
 - Complex instructions: 1–many
 - Microengine similar to RISC
- Comparable performance to RISC
 - Compilers avoid complex instructions

Fallacies

- **Powerful instruction $\Rightarrow$ higher performance**
 - Fewer instructions required
 - But complex instructions are hard to implement
 - May slow down all instructions, including simple ones
 - Compilers are good at making fast code from simple instructions
- **Use assembly code for high performance**
 - But modern compilers are better at dealing with modern processors
 - More lines of code (as is the case for assembly code vs. C or Java) $\Rightarrow$ more errors and less productivity

Fallacies

- Backward compatibility $\Rightarrow$ instruction set doesn't change
 - But they do accrue more instructions



x86 instruction set

Pitfalls

- Forgetting that sequential words are not at sequential addresses
 - Increment by 4, not by 1!
- Keeping a pointer to an automatic variable after procedure returns
 - e.g., passing pointer to a local array/variable back via an argument
 - Pointer becomes invalid when stack popped

Concluding Remarks

- Design principles
 1. Simplicity favors regularity
 2. Smaller is faster
 3. Make the common case fast
 4. Good design demands good compromises
- Layers of software/hardware
 - Compiler, assembler, hardware
- MIPS: typical of RISC ISAs
 - c.f. x86

Concluding Remarks

- Measured MIPS instruction execution frequencies in benchmark programs
 - Consider making the common case fast

Instruction class	MIPS examples	SPEC2006 Int	SPEC2006 FP
Arithmetic	add, sub, addi	16%	48%
Data transfer	lw, sw, lb, lbu, lh, lhu, sb, lui	35%	36%
Logical	and, or, nor, andi, ori, sll, srl	12%	4%
Cond. Branch	beq, bne, slt, slti, sltiu	34%	8%
Jump	j, jr, jal	2%	0%

Assignment 2

- Homework assignment
2.1, 2.5, 2.7, 2.10, 2.32, 2.40
- To be submitted in class on
- Read Chapter 2 of
Advanced Material: Compiling C and
Interpreting Java

实验的时间安排 第11-14周 周六上午 8:30-
11:30